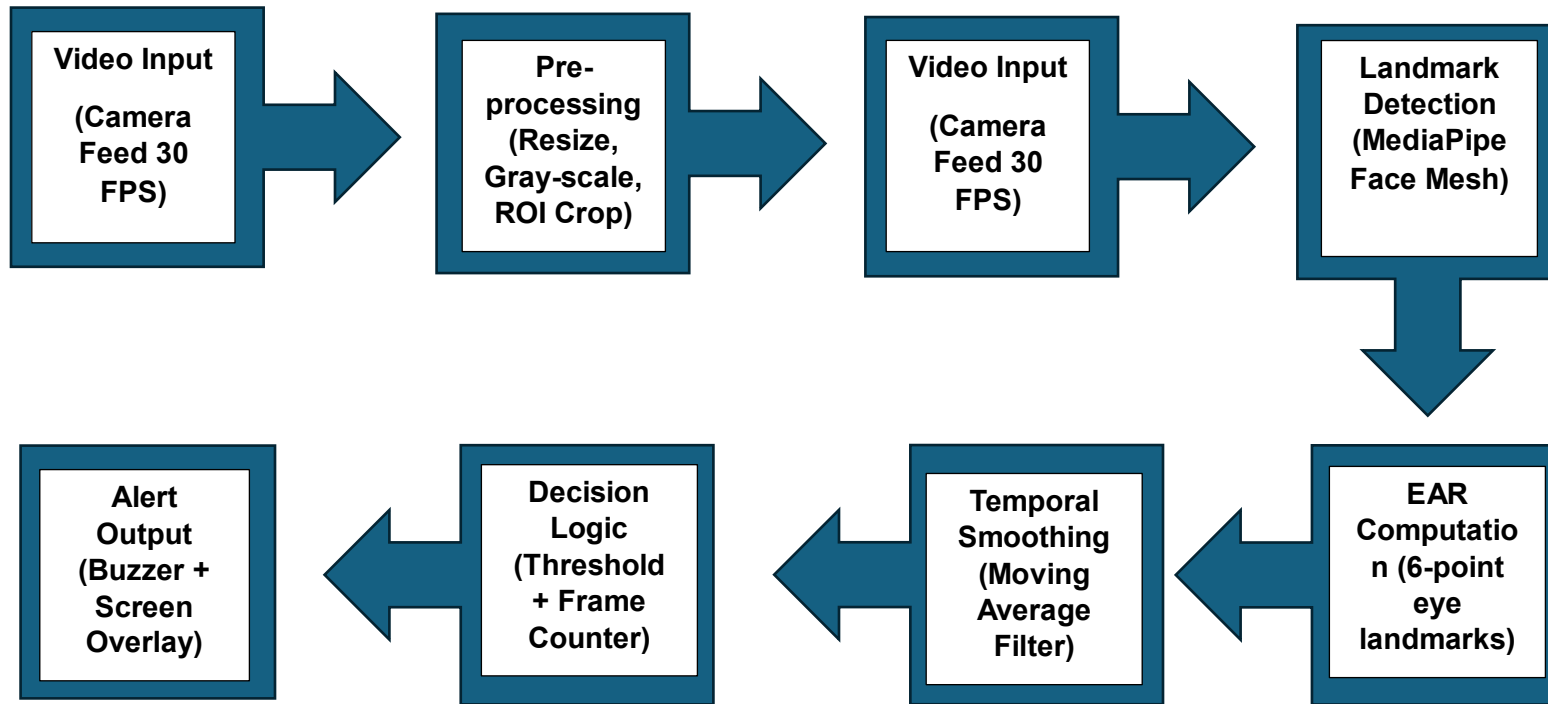


Signal Processing Block Diagram



Signal Flow Description

- Input: Live camera stream captured at 30 FPS via OpenCV VideoCapture
- Pre-processing: Each frame is resized to 640×480, converted to RGB for MediaPipe, and a face bounding-box ROI is cropped to reduce processing area
- Landmark Detection: MediaPipe Face Mesh extracts 468 3D landmarks per frame; 12 specific eye-region points (6 per eye) are selected for EAR calculation
- EAR Computation: $EAR = (\|p2-p6\| + \|p3-p5\|) / (2 \times \|p1-p4\|)$ computed independently for left and right eyes, then averaged

- Temporal Smoothing: A 10-frame moving average filter smooths the EAR signal to suppress blink noise and sensor jitter
- Decision Logic: If smoothed EAR < 0.25 persists for 20+ consecutive frames (~ 0.67 s at 30 FPS), a drowsiness event is flagged
- Alert Output: Buzzer activates and a red on-screen warning overlay is rendered on the live video display

Software Architecture

main.py

- camera_module.py (Real-time video acquisition)
 - OpenCV VideoCapture initialization
 - Frame reading loop
 - Frame resizing and FPS handling

- preprocessing.py (Frame enhancement)
 - RGB conversion
 - Histogram equalization
 - Face ROI extraction

- landmark_detection.py (Facial landmark tracking)
 - MediaPipe Face Mesh initialization
 - Face detection
 - Eye landmark extraction

- ear_calculation.py (Signal extraction)
 - Euclidean distance computation
 - Left eye EAR calculation

- Right eye EAR calculation
- Average EAR generation

- signal_processing.py (DSP processing)
 - Moving average filter
 - Frame buffer management
 - Blink rate computation
 - Noise reduction

- detection_logic.py (Drowsiness classification)
 - Threshold comparison
 - Consecutive frame counter
 - Drowsiness decision logic

- alert_system.py (Warning mechanisms)
 - Audio alarm trigger
 - Visual warning overlay
 - Alert reset logic

- evaluation.py (Testing & analytics)
 - Accuracy computation
 - False positive analysis
 - Latency measurement